

10 Advanced Python Features

A curated collection of **10 standalone Python scripts**, each demonstrating a powerful advanced Python feature with clear, runnable examples.

· Table of Contents

| # | Feature | File | Description | |---|-----|-----|-----| | 1 | **Unpacking** | feature1.py | Basic & extended iterable unpacking, dictionary merging, variable swapping | | 2 | **exec / eval** | feature2.py | Dynamic code execution and safe expression evaluation | | 3 | **Type Annotations** | feature3.py | Variable & function annotations, inspecting `__annotations__` | | 4 | **__repr__ & __str__** | feature4.py | Custom string representations for classes | | 5 | **Decorators** | feature5.py | Writing and applying function decorators | | 6 | **Context Managers** | feature6.py | Custom `__enter__` / `__exit__` for resource management | | 7 | **Custom Iterators** | feature7.py | Implementing `__iter__` and `__next__` protocols | | 8 | **Generators** | feature8.py | Using `yield` for memory-efficient iteration | | 9 | **itertools** | feature9.py | `count`, `cycle`, combinations from the `itertools` module | | 10 | **Async / Await** | feature10.py | Asynchronous coroutines with `asyncio` |

· Getting Started

Prerequisites

- **Python 3.10+** (no external dependencies required)

Running Any Feature

```
python feature1.py    # Replace number with 1-10
```

· Feature Details

Feature 1 · Unpacking (feature1.py)

Demonstrates multiple unpacking techniques:

- **Basic unpacking:** `a, b, c = [1, 2, 3]`
- **Extended unpacking** with `*`: `a, *b, c = [1, 2, 3, 4, 5]`
- **Ignoring values** with `_`
- **Nested structure** unpacking
- **Function argument** unpacking with `*args`
- **List merging** with `[*list1, *list2]`
- **Dictionary merging** with `{**dict1, **dict2}`
- **Variable swapping:** `x, y = y, x`

Feature 2 · exec / eval (feature2.py)

Shows dynamic code execution:

- `exec()` · Execute dynamically constructed code strings in a controlled scope

- `eval()` · Evaluate user-provided mathematical expressions
- **Safe eval** · Restrict evaluation to a limited variable namespace

Feature 3 · Type Annotations (feature3.py)

Covers Python's type hinting system:

- Variable annotations (`name: str, age: int`)
- Function parameter and return type annotations
- Accessing `__annotations__` dictionaries at runtime

Feature 4 · `__repr__` & `__str__` (feature4.py)

Explains the difference between the two dunder methods:

- `__repr__`: Unambiguous, developer-facing representation
- `__str__`: Human-readable, user-facing representation

Feature 5 · Decorators (feature5.py)

Builds a decorator from scratch:

- Wrapper function pattern with `*args, **kwargs`
- `@decorator` syntax sugar
- Before/after execution hooks

Feature 6 · Context Managers (feature6.py)

Custom `FileManager` class implementing:

- `__enter__` · Resource acquisition (open file)
- `__exit__` · Resource cleanup (close file) with exception handling

Feature 7 · Custom Iterators (feature7.py)

`MyRange` class implementing the iterator protocol:

- `__iter__` returns self
- `__next__` yields sequential values and raises `StopIteration`

Feature 8 · Generators (feature8.py)

`count_up_to()` generator function:

- Uses `yield` to pause and resume execution
- Memory-efficient iteration over large ranges

Feature 9 · `itertools` (feature9.py)

Highlights from the `itertools` standard library module:

- `count(start, step)` · Infinite arithmetic counter
- `cycle(iterable)` · Infinite cyclic repetition
- `combinations(iterable, r)` · All r-length combinations

Feature 10 · Async / Await (feature10.py)

Asynchronous programming with asyncio:

- `async def` coroutine definitions
 - `await` for non-blocking I/O simulation
 - `asyncio.run()` as the entry point
-

.. Logic Flow

Each script is self-contained:

1. Define the feature (function, class, or module usage)
 2. Demonstrate with concrete examples
 3. Print output to console for verification
-

· License

Educational project · use freely for learning and reference.